

SecOps Sentinel — Staj Projesi Kapsam ve Görev Dokümanı

Table of Contents

1. Proje Amacı
2. Kapsam
3. Veri Seti
4. Teknoloji Stack
5. Görev Bloğu 1 — Ortam Kurulumu ve Veri Profilleme
6. Görev Bloğu 2 — Veri Modeli ve Ingestion
7. Görev Bloğu 3 — Normalizasyon ve Mesaj Şablonlama
8. Görev Bloğu 4 — Baseline ve Kural Motoru
9. Görev Bloğu 5 — Risk Skolama ve Korelasyon
10. Görev Bloğu 6 — Kural Setinin Tamamlanması ve Yönetim API'leri
11. Görev Bloğu 7 — Performans, Test ve CI
12. Görev Bloğu 8 — Demo Arayüz, Dokümantasyon ve Sunum
13. Genel Teknik Gereksinimler
14. Yaygın Tuzaklar
15. Değerlendirme Kriterleri
16. Ek A — Veritabanı Şeması
17. Ek B — Kural Spesifikasyonları
18. Ek C — Risk Skolama Referansı
19. Ek D — API Endpoint Listesi
20. Ek E — Klasör Yapısı
21. Ek F — Teslim Kontrol Listesi
22. Ek G — MVP Sonrası Kapsam (bilgi amaçlı)
23. Ek H — Referanslar

Proje Amacı

Bir kurumda log toplanıyor olması tek başına değer üretmez. Biri o loglara bakıp 'burada bir sorun var' diyemiyorsa loglar yalnızca disk doldurur. OWASP Top 10:2025 listesindeki A09 kategorisi 2021'de 'Security Logging and Monitoring Failures' iken 2025'te 'Security Logging and Alerting Failures' olarak yeniden adlandırıldı; çünkü asıl eksik olan izleme değil, anlamlı alarm üretebilme yeteneğidir.

Bu projede ham Windows Event Log kayıtlarını alan, her makinenin normal davranışını öğrenen, sapmaları tespit eden, risk skorlayan ve bir güvenlik analistinin bakabileceği önceliklendirilmiş olaylar üreten bir servis inşa edilecek.

- Katmanlı mimariyle production kalitesinde Python backend
- İlişkisel veri modelleme ve performanslı sorgu tasarımı
- Yapılandırılmamış metinden sinyal çıkarma
- İstatistiksel baseline ve anomali kavramları
- Güvenlik operasyonu kavramları: detection, false positive, severity, confidence, MITRE ATT&CK
- İleride bir SPA tarafından tüketilecek API sözleşmesi tasarımı

Kapsam

Kapsam İçi

CSV → Ingestion → Normalizasyon → Baseline → Detection → Scoring → Correlation → REST API

- Kaggle Windows Event Log CSV'sini yükleyen, temizleyen, normalize eden ingestion hattı
- Log mesajlarını şablonlara indirgeyen normalizer
- Makine bazlı davranış temeli (baseline) hesaplayan modül
- 5 kurallı, genişletilebilir detection engine
- Açıklanabilir risk skrolama
- Bulguları olaylara (incident) gruplayan korelasyon katmanı
- Tüm bunları sunan REST API
- API'yi tüketen tek dosyalık demo arayüz (vanilla JS)

Kapsam Dışı

Aşağıdakiler bilinçli olarak dışarıda bırakılmıştır. Zaman kalsa bile mentor onayı olmadan başlanmayacak.

- React uygulaması — MVP'de API sözleşmesi stabilize ediliyor, arayüz sonraki faz
- Canlı log toplama ajanı — ingestion mantığı önce batch üzerinde doğrulanmalı
- Makine öğrenmesi modelleri — baseline istatistiği olmadan ML gürültü üretir
- LLM entegrasyonu — deterministik tespit çalışmadan LLM eklemek hatayı gizler
- Dependency / CVE tarayıcı — ayrı bir üründür
- Kimlik doğrulama ve çok kullanıcıli yetkilendirme — MVP tek kullanıcıli çalışır
- Celery, Redis, Kafka — batch işleme için gerekmiyor

Veri Seti

Verilen dosya: data/raw/windows_event_logs.csv — Windows Event Log (Application/System channel) exportu.

- MachineName — olayı üreten makine; ana gruplama anahtarı
- Category — çoğunlukla (0) gibi anlamsız değerler içerir, düşük sinyal
- EntryType — Information / Warning / Error / SuccessAudit / FailureAudit
- Message — olayın serbest metin açıklaması; asıl bilgi burada
- Source — olayı üreten Windows bileşeni; en değerli kolon
- TimeGenerated — olay zamanı; timezone semantiği belirsiz, netleştirilecek
- country / regionName / city — coğrafi bilgi

Teknoloji Stack

```
Python 3.12
├─ FastAPI          – API katmanı
├─ Pydantic v2      – sözleşme + validation
├─ SQLAlchemy 2.0   – ORM
├─ Alembic          – şema migration
├─ PostgreSQL 16    – veritabanı
├─ pandas           – profillemeye ve batch ingestion
├─ pytest + httpx   – test
└─ ruff + mypy      – lint + tip kontrolü
```

Docker + Docker Compose

GitHub Actions (CI)

Bu listeye ekleme yapmadan önce mentora danışılacak. Listedışı bir kütüphane kullanılacaksa gerekçesi PR açıklamasında belirtilecek. Her bağımlılık bir öğrenme maliyetidir.

Görev Bloğu 1 — Ortam Kurulumu ve Veri Profillemeye

Ortam kurulumu

Repo iskeletini kur, docker-compose ile FastAPI + PostgreSQL ayağa kaldır.

Veri profillemeye raporu

Veri setini notebook'ta analiz et, bulguları docs/data-profile.md dosyasına yaz. Cevaplanacak sorular: satır ve makine sayısı; Source dağılımı ve ilk 30 kaynağın hacim payı; EntryType dağılımı; güvenlikle ilgili kaynakların (Service Control Manager, Application Error, Windows Error Reporting, Microsoft-Windows-Security-Auditing, Windows Defender, EventLog) varlığı; TimeGenerated UTC mi yerel mi ve bu karara nasıl vardığın; kapsanan tarih aralığı ve boşluklar; birebir tekrar eden satır sayısı; bir makinenin birden fazla city değeriyle görünüp görünmediği; Message uzunluk dağılımı; aynı Source altındaki mesajların birbirine benzerliği.

Görev Bloğu 2 — Veri Modeli ve Ingestion

Veri modeli

Ek A'daki şemayı Alembic migration'ları olarak uygula.

CSV ingestion

POST /api/v1/ingest/csv endpoint'i CSV dosyasını alıp veritabanına yazar.

Görev Bloğu 3 — Normalizasyon ve Mesaj Şablonlama

Türetilmiş alanlar

Ingestion sırasında hesaplanacak alanlar.

Mesaj şablonlama

Değişken kısımları yer tutucularla değiştirerek benzer mesajları tek şablona indirge.

Entity extraction

Message içinden yapılandırılmış varlıkları çıkar.

Event API

Event verilerini sunan endpoint'ler.

- DoD: Şablon oranı $\geq 50:1$
- DoD: Şablon extractor için en az 15 unit test (edge case'ler dahil)
- DoD: docs/architecture.md'de normalizasyon stratejisi ve gerekçesi yazılı
- DoD: Event endpoint'leri sayfalı ve filtreli çalışıyor
- DoD: En sık 20 şablon docs/data-profile.md'ye eklendi

Görev Bloğu 4 — Baseline ve Kural Motoru

Baseline hesaplama

Her makine için normal davranış profilini hesapla.

Kural motoru altyapısı

Kuralların ortak arayüz ve registry üzerinden çalıştığı motor.

İlk kurallar: R001 ve R002

Spesifikasyonlar Ek B'de. Bu iki kural motor altyapısının doğru tasarlandığını kanıtlar.

Görev Bloğu 5 — Risk Skorlama ve Korelasyon

Risk skarlama

Her bulguya açıklanabilir bir risk skoru ver.

Korelasyon

Ham bulguları olaylara grupla.

Incident API

Incident yaşam döngüsünü yöneten endpoint'ler.

```
{
  "risk_score": 75,
  "severity": "HIGH",
  "confidence": 0.72,
  "reasons": [
    {"factor": "entry_type_error", "points": 20},
    {"factor": "critical_source:Service Control Manager", "points": 20},
    {"factor": "outside_business_hours", "points": 10},
    {"factor": "new_message_template", "points": 15},
    {"factor": "machine_criticality_high", "points": 15}
  ],
  "mitre_techniques": ["T1489"]
}
```

Görev Bloğu 6 — Kural Setinin Tamamlanması ve Yönetim API'leri

R003, R004, R005

Kalan üç kuralı uygula. Spesifikasyonlar Ek B'de.

MITRE ATT&CK eşleştirmesi

Kuralları uygun MITRE tekniklerine eşle.

Kural yönetimi ve istatistik API'leri

Kuralların çalışma anında yönetilmesini ve dashboard verisinin sunulmasını sağla.

- DoD: 5 kural aktif, en az 3'ü gerçek veride bulgu üretiyor
- DoD: R005 kodda ve API çıktısında is_demo: true olarak işaretli
- DoD: Kural eşiği API'den değiştirilip detection yeniden çalıştırılabilir
- DoD: docs/rules.md her kural için mantık, parametreler, MITRE ve bilinen false positive senaryolarını içeriyor

Görev Bloğu 7 — Performans, Test ve CI

Performans

Sorgu ve işlem sürelerini ölç, iyileştir, belgele.

Test

Anlamlı test kapsamı oluşturun.

CI

Otomatik kalite kapısı kur.

Görev Bloğu 8 — Demo Arayüz, Dokümantasyon ve Sunum

Demo arayüz

Tek bir demo/index.html dosyası. Framework yok, build step yok; sadece HTML + vanilla JS + fetch.

Göstermesi gerekenler: özet kartlar (/stats/overview), severity'ye göre filtrelenebilir incident listesi, incident detayı (bulgular, skorlama gerekçeleri, kanıt event'leri, MITRE teknikleri) ve status değiştirme.

Dokümantasyon

Projenin devralınabilir olmasını sağla.

Sunum

15 dakikalık demo: problem ve OWASP A09 bağlantısı (2 dk), mimari (3 dk), canlı demo — CSV yükle, detection çalıştır, incident incele (6 dk), karşılaşılan en zor problem ve çözümü (2 dk), sınırlılıklar ve sonraki adımlar (2 dk). Slayt üzerinden değil, çalışan sistem üzerinden anlatılacak.

Genel Teknik Gereksinimler

Mimari Disiplin

- NFR-11: Katman ayrımı api/ → services/ → models/; api/ katmanında SQL sorgusu veya iş mantığı olmayacak
- NFR-12: İş mantığı servis katmanında, test edilebilir şekilde
- NFR-13: Konfigürasyon environment'tan, Pydantic Settings ile
- NFR-14: Secret'lar kodda ve git geçmişinde olmayacak

React'e Hazırlık — 8 Zorunlu Madde

Bu fazda React yazılmıyor, ancak API React yazılabilir olacak. Bu maddeler kabul kriterlerindedir ve sonradan düzeltilmesi en pahalı şeylerdir.

- API-first: backend HTML render etmez, Jinja2 yok, tek çıktı JSON
- Tüm liste endpoint'leri aynı zarfı döner: {"items": [...], "total": N, "page": N, "page_size": N}
- Filtre parametreleri tutarlı: ?
machine=&source=&entry_type=&severity=&from=&to=&q=&sort=&page=&page_size=
- Tüm zamanlar UTC + ISO 8601, timezone suffix'li (2020-11-14T08:41:59Z); yerel saate çevirmek frontend'in işi
- Enum değerleri sabit ve büyük harf: CRITICAL, HIGH, MEDIUM, LOW, INFO; serbest metin veya Türkçe dönülmez
- Standart hata formatı: {"error": {"code": "...", "message": "...", "field": "...}}
- CORS yapılandırılmış, izinli origin'ler config'ten okunuyor
- /openapi.json sözleşmedir: tüm Pydantic response modelleri eksiksiz tiplenecek, dict veya Any dönmek yasak

Çalışma Disiplini

- NFR-15: Feature branch + PR; doğrudan main'e commit yok
- NFR-16: Anlamlı commit mesajları; fix, update, asdf kabul edilmez
- NFR-17: PR açıklamasında ne yapıldığı, hangi FR/NFR'ın karşılandığı ve nasıl test edildiği yazılacak
- NFR-18: PR'lar küçük tutulacak, tercihen < 400 satır değişiklik

Yaygın Tuzaklar

Bunlar her yıl tekrarlanan hatalardır. Şimdi oku, ilerde tanı.

- Veriyi görmeden kural yazmak — G1.2 bitmeden detection/'a dokunma
- Timezone'u ertelemek — naive datetime ile başlanırsa her şey yeniden yazılır; TIMESTAMPTZ, hep UTC
- `pd.read_csv(full_file)` — bu bir ingestion çözümü değil; chunk'la, batch insert et
- Router'a SQL yazmak — test edilemez hale gelir, refactor imkansızlaşır
- Eşikleri koda gömmek — `if count > 5` değil, `if count > rule.params['threshold']`
- Finding = Incident sanmak — korelasyon olmadan üretilen 4000 alarm işe yaramaz
- Her bulguya MITRE tekniği yapıştırmak — kanıt yoksa eşleme yapma
- Sentetik veriyi gerçekmiş gibi sunmak — R005'i demo olarak etiketle
- Kapsam dışına kaymak — MVP bitmeden LLM veya ML yok
- Testi sona bırakmak — sonradan yazılan test kodu doğrulamaz, kodun etrafına şekillenir
- Şablonlamayı aceleye getirmek — bu modül zayıfsa R003 çalışmaz ve projenin en özgün kısmı kaybolur

Değerlendirme Kriterleri

- Doğru çalışma — %25: FR/NFR listesi karşılanıyor mu
- Kod kalitesi — %20: katman ayrımı, isimlendirme, tip güvenliği, tekrar yokluğu
- Test — %15: anlamlı testler ve edge case'ler; coverage sayı olarak değil kapsam olarak
- Güvenlik farkındalığı — %15: kuralları savunabiliyor mu, false positive'leri biliyor mu, sınırlılıklar dürüst mü
- Veri anlayışı — %10: profillemeye kural tasarımı gerçekten etkilemiş mi
- Dokümantasyon — %10: başkası kurup çalıştırabiliyor mu
- İletişim — %5: blocker'ları zamanında bildirme, demo netliği

Ek A — Veritabanı Şeması

```
machines(
  id, name UNIQUE, criticality, first_seen, last_seen, event_count
)

message_templates(
  id, template_hash UNIQUE, template_text, param_count,
  first_seen, last_seen, occurrence_count
)

events(
  id, machine_id FK, source, category, entry_type, message,
  template_id FK, ts_utc, hour_of_day, day_of_week, is_business_hours,
  country, region_name, city,
  extracted_entities JSONB, dedup_hash UNIQUE, ingested_at
)

rules(
  code PK, name, enabled, params JSONB, weight,
  mitre_techniques TEXT[], is_demo
)

findings(
  id, rule_code FK, machine_id FK, ts_utc,
  severity, confidence, risk_score, reasons JSONB,
  evidence_event_ids BIGINT[], incident_id FK NULL
)

incidents(
  id, title, status, severity, risk_score, machine_id FK,
  first_seen, last_seen, finding_count,
  mitre_techniques TEXT[], assignee, notes, created_at, updated_at
)

baselines(
  machine_id FK, source, metric, window_start, window_end,
  mean, stddev, sample_count,
  PRIMARY KEY(machine_id, source, metric)
)
```

```
-- Zorunlu index'ler
CREATE INDEX ON events (machine_id, ts_utc DESC);
CREATE INDEX ON events (source, entry_type);
CREATE INDEX ON events (ts_utc DESC);
CREATE INDEX ON events (template_id);
CREATE INDEX ON findings (machine_id, ts_utc DESC);
CREATE INDEX ON incidents (status, severity, first_seen DESC);
```

Ek B — Kural Spesifikasyonları

Parametreler başlangıç değerleridir. Profillemeye bulgularına göre kalibre edilmesi beklenir; değiştirilen her parametrenin gerekçesi docs/rules.md'ye yazılacak.

ERROR_BURST

SERVICE_RESTART_LOOP

NEW_MESSAGE_TEMPLATE

OFF_HOURS_ANOMALY

GEO_INCONSISTENCY (DEMO KURALI)

Roadmap Kuralları (BONUS)

Profillemeye aşamasında ilgili Source değerlerinin veride gerçekten mevcut olduğu doğrulanırsa eklenebilir. İlgili kaynak veride yoksa kural yazılmayacak.

- RARE_SOURCE — makinenin geçmişinde çok nadir görülen provider
- DEFENDER_TAMPER — Defender'ın devre dışı bırakılmasına dair mesajlar → T1562.001
- LOG_CLEARED — event log temizleme kaydı → T1070.001
- CRASH_CLUSTER — aynı modülün tekrarlayan çökmesi

Ek C — Risk Skorlama Referansı

$\text{risk_score} = \min(100, \text{base} + \sum \text{modifiers})$. Aşağıdaki puanlar başlangıç değerleridir, kalibre edilmesi beklenir.

- EntryType = Error: +20
- EntryType = Warning: +10
- EntryType = FailureAudit: +25
- Kritik Source (watchlist'te): +20
- Mesai dışı: +10
- Yeni şablon (R003 tetiklendi): +15
- Makine criticality = HIGH: +15
- Aynı makinede son 24 saatte ≥ 3 bulgu: +10
- 80–100 → CRITICAL
- 60–79 → HIGH
- 35–59 → MEDIUM
- 15–34 → LOW
- 0–14 → INFO

Ek D — API Endpoint Listesi

Tümü /api/v1 altında.

- POST /ingest/csv → FR-01...05
- GET /ingest/jobs/{id} → FR-05
- GET /events → FR-14
- GET /events/{id} → FR-15
- GET /machines → FR-44
- GET /machines/{id} → FR-44
- GET /machines/{id}/timeline → FR-44
- POST /detection/run → FR-23
- GET /incidents → FR-34
- GET /incidents/{id} → FR-35
- PATCH /incidents/{id} → FR-36, FR-37
- GET /rules → FR-40
- PATCH /rules/{code} → FR-41
- GET /stats/overview → FR-42
- GET /stats/timeseries → FR-43
- GET /healthz

```
// GET /stats/overview – beklenen çıktı
{
  "total_events": 148230,
  "total_machines": 12,
  "open_incidents": 7,
  "severity_breakdown": {
    "CRITICAL": 1, "HIGH": 3, "MEDIUM": 2, "LOW": 1, "INFO": 0
  },
  "top_sources": [
    {"source": "Service Control Manager", "count": 4120}
  ],
  "period": {
    "from": "2020-11-01T00:00:00Z",
    "to": "2020-11-30T23:59:59Z"
  }
}
```

Ek E — Klasör Yapısı

```
secops-sentinel/
├─ backend/
│  ├─ app/
│  │  └─ main.py
│  │  └─ core/          # config, logging, exceptions, CORS
│  │  └─ api/v1/        # SADECE routing + validation
│  │  │  └─ events.py    incidents.py    machines.py
│  │  │  └─ rules.py     ingest.py      stats.py
│  │  └─ schemas/       # Pydantic – API sözleşmesi
│  │  └─ models/        # SQLAlchemy
│  │  └─ db/            # session, base
│  │  └─ ingestion/
│  │  │  └─ csv_loader.py  normalizer.py
│  │  │  └─ template_extractor.py
│  │  │  └─ entity_extractor.py
│  │  └─ detection/
│  │  │  └─ engine.py      scoring.py    correlator.py
│  │  │  └─ rules/
│  │  │  │  └─ base.py     registry.py
│  │  │  │  └─ r001_error_burst.py
│  │  │  │  └─ r002_service_restart_loop.py
│  │  │  │  └─ r003_new_template.py
│  │  │  │  └─ r004_off_hours.py
│  │  │  │  └─ r005_geo_inconsistency.py
│  │  └─ baseline/
│  │  │  └─ services/     # iş mantığı burada
│  └─ tests/
│  │  └─ unit/            integration/    fixtures/
│  └─ alembic/
│  │  └─ pyproject.toml
├─ frontend/
│  └─ README.md          # sonraki faz notları
├─ data/
│  └─ raw/               # .gitignore
│  └─ samples/sample_500.csv # repoda
├─ notebooks/01_profiling.ipynb
├─ docs/
│  └─ data-profile.md    rules.md
│  └─ api.md            architecture.md
├─ demo/index.html
├─ docker-compose.yml
├─ .github/workflows/ci.yml
└─ README.md
```

Ek F — Teslim Kontrol Listesi

Fonksiyonel

- Tam CSV hatasız yükleniyor; tekrar yüklemde duplicate yok
- Her event bir template_id ile ilişkili; şablon oranı $\geq 50:1$
- 5 kural çalışıyor; en az 3'ü gerçek veride bulgu üretiyor
- Her bulgunun reasons alanı dolu ve skorla tutarlı
- Bulgular incident'lara gruplanıyor; incident sayısı bulgu sayısından belirgin az
- Incident status API'den değiştirilebiliyor; geçersiz geçiş 400 dönüyor
- Kural parametreleri yeniden deploy olmadan değiştirilebiliyor
- demo/index.html incident listesi ve detayını gösteriyor

Teknik

- docker compose up tek komutla çalışıyor
- React'e hazırlık 8 maddesinin tamamı sağlanıyor
- Hiçbir endpoint dict veya Any dönmüyor; /openapi.json eksiksiz
- Coverage $\geq 70\%$; detection/ ve ingestion/ $\geq 90\%$
- ruff + mypy temiz; CI yeşil
- Tüm zamanlar UTC ve timezone-aware
- api/ katmanında SQL veya iş mantığı yok
- Secret'lar kodda ve git geçmişinde yok
- Performans hedefleri (NFR-01...04) ölçüldü ve karşılandı

Dokümantasyon

- README kurulum, mimari ve ölçülen performans içeriyor
- docs/rules.md her kural için false positive senaryolarını içeriyor
- docs/data-profile.md veri sınırlılıklarını açıkça belirtiyor
- README'de 'Bu sistem neyi tespit edemez?' bölümü var (≥ 5 madde)
- docs/architecture.md açık bırakılmış tüm tasarım kararlarının gerekçelerini içeriyor
- 15 dakikalık demo hazır ve prova edilmiş

Ek G — MVP Sonrası Kapsam (bilgi amaçlı)

- P2 — React dashboard: Vite + TS, openapi-typescript ile üretilmiş tipler, TanStack Query
- P3 — Çok makineli korelasyon, olay zinciri kuralları, MITRE ısı haritası
- P4 — İstatistiksel anomali tespiti: Isolation Forest, STL decomposition
- P5 — Canlı ingestion ajansı, kuyruk altyapısı, WebSocket akış
- P6 — LLM açıklama katmanı: sadece açıklama üretir, tespit ve skorelama deterministik kalır

Ek H — Referanslar

- OWASP Top 10:2025 — <https://owasp.org/Top10/2025/> — projenin doğrudan ilgili olduğu kategori: A09 Security Logging and Alerting Failures
- MITRE ATT&CK Enterprise — <https://attack.mitre.org/> — kullanılan teknikler: T1489, T1562.001, T1070.001
- FastAPI — <https://fastapi.tiangolo.com/>
- SQLAlchemy 2.0 — <https://docs.sqlalchemy.org/>
- Windows Event Log providers — Microsoft Learn